Advanced Data Management, chapters 1–6: every method, formula and command on two pages

blue = topic, green = step-by-step procedure, red ! = the mistake that costs points. Notation: B pages of a table, N or $|R|$ rows, r rows per page, M buffer pages available to an operator (`work_mem` / 8 KB: 4 MB = 512 pages), h tree height, F fanout, L entries per leaf, s selectivity (fraction of rows kept), d distinct values, k matching rows. Example table used in illustrations: `orders` 500,000 rows, 3,185 pages, 157 rows/page; `customers` 10,000 rows, 73 pages.

1 Architecture and the life of a query

Five subsystems: connection/session manager (one worker per connection) ▹ query processor (parser, binder, rewriter, optimiser = *planner*, executor) ▹ storage manager (files/pages, access methods = heap + indexes, *buffer pool*) ▹ transaction manager (locks, log, recovery) ▹ catalog (schema + statistics: `pg_class`, `pg_stats`). **Symptom → owner**: syntax error: parser. "column does not exist", type, permission: binder (catalog). Seq Scan where an index is cheaper, bad join order: planner. Estimate 1,000× off: statistics (`ANALYZE`). Low hit ratio: buffer pool. Lost update, deadlock: transaction manager. Spilled sort: `work_mem`. **Five stages, five objects**: parse (grammar only → parse tree) → bind (names, types, permissions against the catalog → bound tree) → rewrite (relational algebra, *logical*: views inlined, subqueries flattened, constants folded, selection/projection pushdown → logical plan) → optimise (access path, join algorithm, join order, by cost from statistics, *physical* → plan tree) → execute (iterators → rows). **Algebra**: σ_p select rows, π_A project columns, $\bowtie$ join, γ group. $\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S$ if p is on R only (push selection); projection pushed likewise (drop unused columns before the join); $R \bowtie S = S \bowtie R$, $(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$: join order is free, so the optimiser must search it. **Iterator model**: every node has `open/next/close`; `next` requests go down, tuples come up one at a time, pipelined. **Streaming**: Seq/Index Scan, Filter, Project, Limit, Nested Loop, Merge Join, hash *probe*, GroupAggregate on sorted input. **Blocking** (pipeline breaker: consumes all input before the first output): Sort, Hash (build side), HashAggregate, Materialize. Below a breaker nothing above it starts; a breaker may spill to disk (`Disk`, `Batches`). **SQL evaluation order**: FROM/JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. So: a SELECT alias works only in ORDER BY; a condition on an aggregate goes in HAVING; every selected column is grouped or aggregated; WHERE filters rows, HAVING filters groups. **OLTP**: many short transactions, whole rows by key: row store, B-trees, high hit ratio. **OLAP**: few long scans of few columns: column store (reads $1/k$ of the pages for 1 of k columns, compresses well; one row update touches k places), Parquet, BRIN.

! "The parser catches a misspelled column" (binder). "The executor chooses the scan" (planner). "Hash Join is blocking" (its build child is; the join streams probes). Alias in WHERE. Aggregate in WHERE.

2 Storage: pages are the cost

Hierarchy: L1 1 ns ▹ RAM 100 ns ▹ SSD 100 μs ▹ HDD 10 ms: disk is 10^3–10^5× RAM, so count *pages* (8 KB), not instructions. Sequential (≈ 16 μs/page at 500 MB/s) beats random (one seek per page: SSD 100 μs, HDD 10 ms).

cost = pages read + pages written — CPU ignored unless data is cached (then CPU-bound)

time = pages × per-page cost of the device and pattern — 3,185 p: 51 ms seq, 0.3 s SSD random, 32 s HDD random

record = sum of column bytes + 24 header, rounded up to a multiple of 8 — PostgreSQL tuple; variable-length columns add a null bitmap, big values go out of line (TOAST)

$r = \lfloor (8192 - 24)/(\text{record} + 4) \rfloor$ — rows per page; +4 = the slot; floor, rows never straddle pages

$B = \lceil N/r \rceil$, size = $8B$ KB — compare with `pg_relation_size`; a gap = dead tuples or TOAST

Slotted page: header (free-space pointers), slot directory (offset, length per tuple) growing down, tuples growing up from the end. Row id = (page, slot) = `ctid`; the slot indirection lets a tuple move inside its page (compaction) without touching any index, and supports variable-length rows. Fixed-length rows: position computable; variable-length: needs the slot offsets.

	heap	sorted	hashed
scan all	B	B	B
= present	$B/2$ (B worst)	$\lceil \log_2 B \rceil$	≈ 1
= absent	B	$\lceil \log_2 B \rceil$	≈ 1
range	B	$\log_2 B$+ pages	B
insert	1 (append)	up to B (shift)	1

Choose: equality → hashed; ranges, ordered output → sorted; append-only, scans → heap. Real systems: heap + indexes. Column store: 1 column of k costs B/k pages. **PostgreSQL**: table = heap file of 8 KB pages; DELETE/UPDATE leave dead tuples (`n_dead_tup`); **VACUUM** frees them inside pages and truncates only empty *trailing* pages (file size unchanged after a scattered delete); **VACUUM FULL**/CLUSTER rewrite the file (size ∝ live rows; CLUSTER also orders it by an index → correlation 1). **CPU-bound, not I/O-bound**: all hits, reads ≈ 0, second run no faster (regex, jsonb, arithmetic on cached data).

! Counting bytes or rows instead of pages. Forgetting the slot or the floor. "Absent key costs $B/2$" (it is B). "Hashing helps ranges". "VACUUM shrinks the file". "Column stores are always better" (worse for one-row reads/writes by k). Trusting a fast second run (cache).

3 Indexes

An index = a separate structure mapping key value → row id (`ctid`), kept beside the heap; every index entry ≈ key +6 B id (≈ 16 B/row).

#leaves = $\lceil N/L \rceil$, $h = \lceil \log_F N \rceil$ — find h: smallest h with $F^h \geq N$; typical 3–4 (F 100–500); never $\log_2$

point lookup = $h + 1$ — h index pages +1 heap page; absent key: h; top levels are cached: often 1–2 real I/Os

range with k matches = $h + (\lceil k/L \rceil - 1) + \text{heap}(k)$ — heap(k): k if secondary (one random read per match), $\lceil k/r \rceil$ if clustered, 0 if index-only

time = ∑ reads × device latency — e.g. 5 reads: 0.5 ms SSD, 50 ms HDD

B$^+$ tree: routing keys inside, all entries in the leaves, leaves sorted and chained (range scan = descend once, walk right), one page per node, always balanced. **Insert**: descend, insert in the leaf; if full, **split**: keep the lower half, move the upper half to a new leaf, chain it, push the first key of the new leaf into the parent; a full parent splits too; a root split adds a level. **Delete**: remove; a leaf under half full borrows from a sibling or merges. Cost $O(h)$. **Clustered**: heap in index order, one per table, ranges cost $\lceil k/r \rceil$ pages; **secondary**: any number, matches scattered, k random pages. **Kinds**: **B-tree** (default; =, <, BETWEEN, ORDER BY, LIKE 'abc%'); **hash** (= only, no order); **bitmap index** (a bit vector per value, low-cardinality analytics, AND/OR by bit ops; PostgreSQL builds bitmaps on the fly from a B-tree); **BRIN** (min/max per block range, tiny, physically ordered columns); **GIN/GiST** (jsonb, text, geometry). **Composite** `(a,b)`: sorted by `a`, then `b` within equal `a`: only a *leading prefix* descends (`a`; `a AND b`; `a=.. AND b>..`); `b` alone: no (scan of the index at best); put equality columns first, the range column last, the most selective equality first. **Expression** `(lower(email))` serves `WHERE lower(email)=..` only. **Partial** `WHERE status='open'`: small, used only when the query repeats the predicate. **Covering** `INCLUDE (cols)`: index-only scan, `Heap Fetches: 0` after VACUUM (visibility map). **Unique** = constraint.

plain index scan ≈ $h + \lceil sN/L \rceil + 4\,sN$ (random heap reads) — vs Seq Scan B; cost units: random = 4× sequential

heap pages touched = $B\,(1 - (1 - s)^r)$ — expected; $sr \geq 3 \Rightarrow$ essentially every page, in random order

index wins iff $4sN < B \iff s < 1/(4r)$ — 157 rows/page → $s < 0.16\%$; 50 rows/page → 0.5%

Bitmap scan: collect row ids, sort by page, read each heap page once in physical order: wins between $1/(4r)$ and a few percent; `Heap Blocks: exact=` counts the pages.

Correlation (`pg_stats.correlation`, physical order vs key order): ±1 (append-only, after CLUSTER): a range of s rows touches ≈ sB pages, cheap index ranges and BRIN; ≈ 0: use the $(1 - (1 - s)^r)$ formula, index useless for large s.

Price: space; every INSERT writes 1+ (number of indexes) random pages (each 2–4× slower); every UPDATE of an indexed column = delete + insert; more plans to cost. **Keep**: PK, FK/join columns, selective filter and frequent ORDER BY columns; one composite over several singles. **Drop**: `idx_scan = 0`, columns with $s \gg 1/(4r)$ (status 90% one value, booleans).

! $\log_2 N$ for the height. Forgetting the +1 heap read. Counting matching rows, not pages. "I created it, so it is used". (a, b) serving `b` alone. Hash or BRIN for a range / ORDER BY. Indexing `name` and querying `lower(name)`.

4 Sorting and joins

External merge sort, B pages, M buffers: pass 0 reads M pages at a time, sorts in memory, writes $\lceil B/M \rceil$ runs; each merge pass merges $M - 1$ runs (1 buffer is the output) into one.

runs = $\lceil B/M \rceil$, fan-in = $M - 1$

merge passes = $\lceil \log_{M-1} \text{runs} \rceil$ — procedure: divide runs by $M - 1$, ceil, repeat until 1; count divisions

total passes = 1 + merge passes, I/O = $2B$ × total passes — every pass reads and writes all B (PostgreSQL skips the final write when streaming up)

one pass $\iff M \geq B$; two passes $\iff \lceil B/M \rceil \leq M - 1$ ($M \gtrsim \sqrt{B}$) — smallest M for 2 passes: solve $M(M-1) \geq B$

Doubling M removes at most one merge pass (never "halves the time"). **PostgreSQL**: `Sort Method: quicksort Memory: xkB` = fit; `external merge Disk: xkB` = spilled, raise `work_mem` (per sort/hash *per connection*: `SET LOCAL` in a transaction); `top-N heapsort` = ORDER BY + LIMIT. **Joins** (R outer/left/probe, S inner/right/build; $B_R \leq B_S$ unless said otherwise):

simple NL	$B_R + \lvert R \rvert\, B_S$ (rows of R!)
block NL	$B_R + \lceil B_R/(M-2) \rceil\, B_S$
outer fits ($B_R \leq M - 2$)	$B_R + B_S$
index NL	$B_R + \lvert R \rvert (h + m)$, $m = \lvert S \rvert / d_S$
sort-merge	$\text{sort}(R) + \text{sort}(S) + B_R + B_S$
inputs already sorted	$B_R + B_S$
hash, build fits in M	$B_R + B_S$ (build = smaller)
Grace (partitioned) hash	$3(B_R + B_S)$ (`Batches`>1)

Procedure: put the smaller table outside (NL) / as build (hash); check whether it fits in $M - 2$ or M; compute each candidate; the sort term is the full external-sort I/O of that input unless it is already sorted or indexed. Hash join needs $B_{\text{build}} \leq M$ in memory (else partition both by hash of the key into $M - 1$ partitions, write, then join partition pairs: the 3×). Partition skew (one hot key) breaks hashing. **Which join**: **Block NL**: one input fits, *any* condition (inequality, LIKE) / disaster: two big inputs, nothing fits. **Index NL**: tiny outer (point query, selective filter) + index on the inner key, ≈ $|R| \times (h+1)$ random reads / disaster: big outer (`loops=` huge, each loop re-descends). **Sort-merge**: inputs already sorted, sorted output wanted (ORDER BY/GROUP BY on the key), range conditions, little memory / disaster: nothing sorted and memory small (two full sorts). **Hash**: big equi-join, build fits in `work_mem`, no order / disaster: non-equality, build ≫ M (batches), skew. Keep all three: each is best somewhere. `enable_*=off` adds a 10^{10} penalty (not a ban).

Aggregation: **HashAggregate** (hash on the group key; few groups, must fit; breaker) vs **Sort + GroupAggregate** (many groups, or input already sorted by an index/merge join; streams group by group). DISTINCT = the same choice; ORDER BY = a Sort unless an index gives the order.

! $|R|$ where B_R belongs. Fan-in M (it is $M - 1$), block NL with M (it is $M - 2$). Forgetting pass 0. Counting reads only ($2B$ per pass). Large table outside a nested loop. Hash join on <. `actual time=a..b` read as a duration (first row .. last row). "Every aggregate blocks".

5 Optimisation: estimates, costs, search

Statistics (`ANALYZE` samples 30,000 rows): per table `reltuples`, `relpages`; per column `null_frac`, `n_distinct` (< 0: a fraction of the rows), `most_common_vals/freqs` (MCV), `histogram_bounds` (100 equi-depth buckets of the non-MCV values, 1% each), `correlation`.

col = v, $v \in$ MCV : $s = \text{freq}(v)$ — read it off the list

col = v, $v \notin$ MCV : $s = \dfrac{1 - \text{null_frac} - \sum \text{MCV freqs}}{\text{n_distinct} - \#\text{MCV}}$ — no MCV list: $s = 1/d$ (uniform); unknown d: default 0.5%

col < c : s = (buckets below c + share of c's bucket)/100 — 100 buckets of 1%; BETWEEN a, b: frac(b) − frac(a); >: 1 − frac; no histogram: default 1/3

LIKE 'abc%' : prefix range; col ≠ v : $1 - s_=$; IS NULL : null_frac

$s_{A \wedge B} = s_A s_B$, $s_{A \vee B} = s_A + s_B - s_A s_B$, $s_{\neg A} = 1 - s_A$ — independence assumption; correlated AND: product *under*-estimates (true s is up to $\min(s_A, s_B)$)

rows = sN, pages = $\lceil \text{rows}/r \rceil$ or $B(1 - (1 - s)^r)$

$|R \bowtie S| = \dfrac{|R|\,|S|}{\max(d_R, d_S)}$ — key–FK join: = |FK side| after its filters; matches per outer row $m = |S|/d_S$

errors multiply : $\text{est}_{\text{join}} = \text{est}_R \times \text{est}_S / \max(d)$ — a leaf off by x makes every node above off by ≥ x: fix the *lowest* wrong node

q-error = max(est/act, act/est) — < 2 fine, ≥ 10 a red flag, ≥ 100 the cause of the plan

Cost model (units = sequential page reads, not ms): `seq_page_cost` 1, `random_page_cost` 4 (1.1 on SSD), `cpu_tuple_cost` 0.01, `cpu_index_tuple_cost` 0.005, `cpu_operator_cost` 0.0025.

Seq Scan = $B + 0.01N + 0.0025\,qN$ — q = predicates per row; e.g. orders: 3,185 + 5,000(+1,250)

Index Scan ≈ $4h + 4 \cdot \text{pages}(k) + 0.015\,k$ — per lookup; unclustered: pages(k) = k; clustered: k/r

Bitmap scan ≈ index leaves + pages(k) (seq . . . random) + $0.01k$

Hash Join ≈ cost(R) + cost(S) + $0.0025(|R|+|S|) + 0.01\,|\text{out}|$ — Nested Loop: cost(R) + $|R|$× cost(inner per loop)

cost(plan) = ∑ nodes, bottom-up, on *estimated* sizes — `cost=start..total`: start = before the first row (a breaker's whole input)

Access path: index iff $s < 1/(4r)$ (adjust by correlation); bitmap up to a few %; else Seq Scan. **Join order**: n tables: $n!$ left-deep orders, more bushy; **System-R DP**: best plan per subset, level by level, $2^n - 1$ subsets, keeping one extra plan per *interesting order* (a sort order a later merge join or ORDER BY can use); beyond `join_collapse_limit` (8) / `geqo_threshold` (12) heuristics or genetic search, written order frozen. **Rule-based** (fixed priorities, no data) vs **cost-based** (statistics; wrong only when the statistics are).

Three failures and fixes: (1) **stale statistics** (`n_live_tup` ≠ `reltuples`, `last_analyze` old, table grew 10×) →

ANALYZE t. (2) **correlated columns** (a AND b where a determines b: product ≪ truth) → CREATE STATISTICS s (dependencies) ON a, b FROM t; ANALYZE t. (3) **skew** (one value 90%, a rare value estimated at $1/d$) → ALTER TABLE t ALTER COLUMN c SET STATISTICS 1000; ANALYZE t (longer MCV list, finer buckets); also CREATE STATISTICS (ndistinct), (mcv) on column pairs. **Why estimation is the hard part**: the search (DP) is solved; every later decision (algorithm, order, memory, index) is a function of row counts, and a wrong leaf multiplies through the tree. **Levers**, no hints: statistics → cost constants (random_page_cost, effective_cache_size) → work_mem → enable_* for experiments → rewrite the query → pg_hint_plan last.

! Adding selectivities for AND. min or $d_R + d_S$ in the join size. Forgetting a cost term. Treating cost as ms. Fixing a 1,000× estimate with an index or enable_hashjoin=off (symptom, not cause). Reading the inner side's actual rows as a total (it is per loop). $n!$ as "number of plans".

6 Buffer management

Pool = array of 8 KB **frames** + page table (page id → frame). Per frame: **pin count** (operators using it now; > 0 cannot be evicted), **dirty bit** (modified since read; must be written before the frame is reused), usage_count. **Read path**: look up → hit: pin, return / miss: pick a victim (unpinned), write it if dirty (2nd I/O), read the page, pin. First touch of any page is a compulsory miss.

EAT = $h\,T_{\text{hit}} + (1-h)\,T_{\text{miss}}$ RAM 100 ns, SSD 100 μs, HDD 10 ms: the miss term dominates unless $h \to 1$; 99% on HDD ≈ 100 μs = 1,000× RAM

h = hit/(hit + read) from Buffers: or pg_statio_user_tables (heap_blks_hit/read); OLTP wants 99.9%+

read = not in shared_buffers: served by the *OS page cache* or the disk (why cold/warm gaps are often small). temp read/written = a sort or hash spilling, not the pool. dirtied, written = pages modified / flushed by this query. **Why its own pool, not the OS cache**: it knows the access pattern (scan = use once → ring; OS LRU floods), the write order (log before data page: WAL), what is pinned, and its memory budget (no double buffering), and it can prefetch by plan. **Policies**: **OPT** (evict the page used furthest in the future; yardstick) ▸ **FIFO** (by load time, a hit changes nothing; cheap; *Belady's anomaly*: more frames can mean more misses) ▸ **LRU** (by last use; a hit moves the page to the front; stack property: more frames never hurt; costs a list move per hit under a lock) ▸ **Clock** (circular, one reference bit; the hand clears bits and evicts the first 0: LRU approximation without the list) ▸ **PostgreSQL clock-sweep** (usage_count 0–5, +1 per hit, hand decrements, evict at 0: recency + frequency) ▸ **MRU** wins for a loop larger than the pool. **Sequential flooding**: one scan of $S \gg$ pool pages under LRU streams use-once pages through every frame and evicts the whole hot working set W; the next transactions pay W misses to reload it; the scan itself gains nothing from caching (all compulsory). **Ring buffers**: a Seq Scan of a table larger than shared_buffers/4 cycles through a private 256 KB ring (32 frames), so W survives if $W \le N - 32$; COPY/CTAS use a 16 MB ring, VACUUM 256 KB; index scans never ring. **Procedure**: compare B with shared_buffers/4 (128 MB → 16,384 frames → threshold 4,096 pages): below, the table stays resident after one scan; above, it is ringed and every scan reads again. **Working set**: h climbs steeply until the pool holds the hot set, then plateaus: size for the working set, not the database; a streaming workload stays at $h \approx 0$ at any size. **Dirty pages**: written by the background writer, at checkpoints, or at eviction. **Steal** (a dirty page of an *uncommitted* transaction may be written) needs *undo*; **no-steal** needs memory for all uncommitted work. **Force** (commit writes all its data pages: slow, random) vs **no-force** (commit does not) needs *redo*. Real systems: **steal + no-force + WAL**: log record written *before* its data page (write-ahead), log forced to disk at commit (one sequential append), data pages later; crash: redo committed records, undo uncommitted. Committed data is safe in the log, not necessarily in the data file. **Sizing**: shared_buffers ≈ 25% of RAM (default 128 MB); more starves the OS cache (which serves the reads) and work_mem per connection, and makes checkpoints bursty; effective_cache_size = pool + OS cache, for the planner. Measure h and the working set first.

! "read = disk". "LRU always beats FIFO" (not on a scan larger than the pool). "95% is a good hit ratio" (compute EAT). Pin/usage/dirty confused. "No-force loses committed data" (the log has it). "Commit writes the pages" (it writes the log). Tracing LRU as FIFO.

7 Reading a plan

Node line: (cost=start..total rows=est width=bytes) = the planner's guess; (actual time=first..last rows=act loops=n) = measured, *per loop*; total rows = act × loops; time = last × loops. **Sub-lines**: Filter + Rows Removed by Filter (tested per row after reading); Index Cond (answered inside the index); Recheck Cond + Heap Blocks: exact= (bitmap scan; pages touched); Heap Fetches (index-only; 0 is ideal); Hash Cond/Merge Cond/Join Filter; Buckets Batches Memory Usage (hash build; Batches> 1 = spilled); Sort Key + Sort Method (+ Memory/Disk); Group Key; Buffers: shared hit/read/dirtied/written, temp read/written (per node, children included); Planning Time, Execution Time; Workers. **Tree**: indentation = children; first child = outer/probe, second = inner/build (the one under Hash); read bottom-up (leaves first), execution starts at the leftmost leaf of the deepest breaker.

Five-move method

1 Leaves: scan type per table, filter, est vs act. ▸ 2 Walk up: join algorithm, join order, aggregate, sort. ▸ 3 Find the lowest node with a large q-error: that is the cause (everything above inherits it). ▸ 4 Resources: Buffers (hit ratio, read), Sort Method, Batches, loops on the inner side, Heap Blocks. ▸ 5 Decide: wrong plan because of estimates (statistics fix) / right plan but slow (memory, index, cold cache, CPU-bound) / nothing to fix; name the command. **Reconstruct the query**: Seq Scan on t a = FROM/alias; Hash/Merge Cond = ON; Filter/Index Cond = WHERE; Group Key = GROUP BY; Sort Key = ORDER BY (an aggregate in parentheses = sort on a computed column); Limit = LIMIT; Aggregate with no key = count(*). **EXPLAIN q**: plan + estimates only, nothing runs (safe on production; shows what the planner *believes*). **EXPLAIN ANALYZE q**: runs it, adds actual rows/time/loops, Sort Method, Batches, Execution Time (DML: wrap in BEGIN; ... ROLLBACK;). **EXPLAIN (ANALYZE, BUFFERS)**: adds hit/read/temp per node: the only way to see the pool. Options: TIMING OFF, FORMAT JSON, VERBOSE (output columns).

8 Instruments and SQL to type

Size and pages

```
SELECT reltuples::bigint, relpages FROM
  pg_class WHERE relname='t'; -- what the
  planner believes
SELECT pg_relation_size('t') / 8192 AS pages;
  SELECT pg_size_pretty(
  pg_total_relation_size('t') ),
  pg_size_pretty( pg_indexes_size('t') ); --
  real file; total = heap + indexes + TOAST
CREATE EXTENSION pageinspect; SELECT count(*)
  FROM heap_page_items(get_raw_page('t', 0));
  SELECT ctid, * FROM t LIMIT 3; -- ctid =
  (page, slot); t_xmax<>0 = dead
VACUUM (VERBOSE) t; VACUUM FULL t; CLUSTER t
  USING idx; SELECT n_live_tup, n_dead_tup,
  last_analyze, last_autoanalyze FROM
  pg_stat_user_tables WHERE relname='t';
```

Statistics

```
\x SELECT attname, null_frac, n_distinct,
  most_common_vals, most_common_freqs,
  histogram_bounds, correlation FROM pg_stats
  WHERE tablename='t' AND attname='c';
ANALYZE t; ANALYZE t (c); VACUUM ANALYZE t;
  SHOW default_statistics_target; -- 100
ALTER TABLE t ALTER COLUMN c SET STATISTICS
  1000; ANALYZE t;
CREATE STATISTICS s_ab (dependencies,
  ndistinct, mcv) ON a, b FROM t; ANALYZE t;
  SELECT * FROM pg_stats_ext;
```

Steering and experiments

```
SET work_mem = '64kB'; SET enable_hashjoin =
  off; SET enable_seqscan = off; SET
  random_page_cost = 1.1; RESET work_mem; RESET
  ALL; SHOW shared_buffers;
BEGIN; SET LOCAL work_mem = '256MB'; EXPLAIN
  (ANALYZE, BUFFERS) ...; COMMIT; -- per
  statement, never globally
EXPLAIN q; EXPLAIN ANALYZE q; EXPLAIN (ANALYZE,
  BUFFERS) q; BEGIN; EXPLAIN ANALYZE DELETE
  ...; ROLLBACK;
```

Buffer pool

```
CREATE EXTENSION pg_buffercache; SELECT
  c.relname, count(*) AS frames, count(*)
  FILTER (WHERE isdirty) AS dirty FROM
  pg_buffercache b JOIN pg_class c ON
  b.relfilenode = pg_relation_filenode(c.oid)
  GROUP BY 1 ORDER BY 2 DESC;
SELECT heap_blks_hit, heap_blks_read,
  round(100.0 * heap_blks_hit /
  nullif(heap_blks_hit + heap_blks_read, 0), 1)
  AS hit_pct FROM pg_statio_user_tables WHERE
  relname='t';
```

Indexes

```
CREATE [UNIQUE] INDEX name ON t (a, b); ON t
  (lower(email)); ON t (user_id) WHERE
  status='open'; ON t (a) INCLUDE (b); ON t
  USING brin (ts); ON t USING hash (k); ON t (a
  DESC, b); DROP INDEX name;
SELECT indexrelname, idx_scan, idx_tup_read
  FROM pg_stat_user_indexes WHERE relname='t';
  SELECT indexdef FROM pg_indexes WHERE
  tablename='t'; \d t \timing on
```

Columns out

```
COPY (SELECT ts FROM t) TO '/data/ts.parquet'
  (FORMAT parquet); -- DuckDB: SELECT ts::date
  AS d, count(*) FROM
  read_parquet('/data/ts.parquet') GROUP BY d
  ORDER BY d;
```

Query patterns (evaluation order in §1)

```
SELECT u.country, count(*) AS n, sum(o.amount)
  AS rev, avg(o.amount), count(DISTINCT
  o.user_id) FROM orders o JOIN users u ON u.id
  = o.user_id WHERE o.status = 'paid' AND o.ts
  >= now() - interval '30 days' GROUP BY
  u.country HAVING count(*) > 100 ORDER BY rev
  DESC LIMIT 10;
LEFT JOIN (keep all left rows, NULL right);
  WHERE id IN (SELECT ...); WHERE EXISTS
  (SELECT 1 FROM o WHERE o.uid = u.id); WITH x
  AS (...) SELECT ... FROM x; count(*) FILTER
  (WHERE c='a'); CASE WHEN .. THEN .. ELSE ..
  END; v::date date_trunc('day', ts)
  extract(dow FROM ts) coalesce(x, 0)
UPDATE t SET c = v WHERE ...; DELETE FROM t
  WHERE ...; INSERT INTO t (a, b) VALUES (..),
  (..); INSERT INTO t SELECT ...; CREATE TABLE
  t (id serial PRIMARY KEY, uid int REFERENCES
  u(id), ts timestamptz);
```

Generate test data (one random() draw per row, several cuts; then ANALYZE)

```
INSERT INTO events (user_id, ts, kind, amount)
  SELECT 1 + (random()*999999)::int, now() -
  random()*interval '365 days', CASE WHEN r <
  0.90 THEN 'view' WHEN r < 0.995 THEN 'cart'
  ELSE 'purchase' END, round((5 +
  random()*100)::numeric, 2) FROM (SELECT
  random() AS r FROM generate_series(1,
  1000000)) g; ANALYZE events;
generate_series(1,n) AS i; i % 10 (cycling);
  (random()*n)::int (uniform);
  floor(-ln(random())*m) (long tail);
  md5(i::text), 'x' || i (strings); now() - (i
  || ' s')::interval (ordered ts, correlation
  1)
```

! random() inside each WHEN (a new draw per branch: wrong proportions). Alias in WHERE. EXPLAIN ANALYZE on DML outside BEGIN/ROLLBACK. CREATE STATISTICS/SET STATISTICS without the ANALYZE after. SET work_mem globally. Forgetting ANALYZE after a bulk load.

9 Procedures for every question type

Storage estimate

record = columns +24 → round to 8 ▸ $r = \lfloor 8168/(\text{record}+4) \rfloor$ ▸ $B = \lceil N/r \rceil$, size = $8B$ KB ▸ scan = B, no-index lookup $B/2$ (present) / B (absent) ▸ seconds = $B\times$ device per-page cost.

Index decision (does this index help?)

s from the predicate (statistics rules of §5) ▸ $k = sN$ rows ▸ pages touched: correlation ≈ 0: $B(1-(1-s)^r)$ (if $sr \ge 3$: all B); correlation ≈ ±1: ≈ sB ▸ index cost ≈ h + leaves + 4× pages touched vs Seq Scan B ▸ rule: plain index iff $s < 1/(4r)$; bitmap up to a few %; else no ▸ add the write price (one random page per insert per index) and say yes/no in one sentence.

B^+ tree cost

$h = \lceil \log_F N \rceil$ (bracket N between powers of F) ▸ lookup $h+1$ ▸ range $h + \lceil k/L \rceil - 1$ + heap(k) ▸ multiply reads by the device latency; note which levels are cached.

External sort

runs = $\lceil B/M \rceil$ ▸ merge passes: divide by $M-1$ with ceil until 1 ▸ I/O = $2B(1+\text{passes})$ ▸ "what changes with M'": recompute; one fewer pass at most per doubling ▸ in PostgreSQL: Sort Method line, fix with work_mem ≥ the Disk: figure.

Join cost comparison

identify B_R, B_S, $|R|$, $|S|$, M, d, indexes, existing order ▸ small input outside/build ▸ compute every candidate from the table in §4 (sort terms as full external sorts unless sorted) ▸ pick the minimum, name the trap answers (rows for pages, M for $M-2$) ▸ say when the loser would win.

Selectivity and estimated rows

per predicate: MCV freq / $1/d$ / histogram fraction ▸ combine: AND product, OR inclusion–exclusion, NOT complement ▸ rows = sN ▸ join: $|R||S|/\max(d)$, key–FK = FK side ▸ compare with actual: q-error; locate the lowest wrong node ▸ name the failure (stale / correlation / skew) and the exact command.

Plan diagnosis

the five moves of §7 ▸ quote the node, est vs act, loops, Buffers, Sort Method/Batches ▸ give the mechanism (why the planner chose this given its estimate) ▸ fix in order: ANALYZE → extended statistics → work_mem/index → rewrite; re-run EXPLAIN (ANALYZE, BUFFERS) to verify.

Buffers line

hit + read = pages requested; compare with B (all of the table? once or per loop?) ▸ h = hit/(hit+read) ▸ cold vs warm: read ≠ disk (OS cache); CPU work identical ▸ temp = spill, raise work_mem ▸ ringed? compare B with shared_buffers/4.

Replacement trace

draw the frames ▸ per request: hit (LRU: move to front; FIFO: nothing; Clock: set bit) or miss (victim by rule, dirty ⇒ extra write) ▸ count hits, misses, writes ▸ EAT = $hT_h + (1-h)T_m$ ▸ flooding: hot set W survives iff the scan is ringed and $W \le N - 32$.

Ticket / discussion answer

claim ▸ mechanism (subsystem, rule) ▸ a number or formula with units ▸ consequence, recommendation ▸ the tempting wrong answer. **Quick check**: letter + one number. **Situation**: data → formula → units → decision.